

Day 11: Advanced Java Concepts (20-June-2026, Saturday)

Session Time: 9:00 AM – 1:00 PM (No lunch break; Tea Break: 10:30 AM – 10:45 AM)

Note: Saturday sessions are shorter (4 hours instructional). Focus on core concepts with condensed hands-on activities.

Session Objectives

By the end of Day 11, learners will be able to:

- Implement **Generics** for type-safe collections and classes.
- Understand **Wrapper Classes** and autoboxing/unboxing.
- Design **Immutable Classes** with proper encapsulation.
- Implement **Singleton Design Pattern** (eager, lazy, thread-safe).
- Apply **SOLID Principles** to write maintainable and scalable code.
- Solve LeetCode: *LRU Cache* (using LinkedHashMap) and *Valid Sudoku* (using HashSet).
- Solve HackerRank: *Java Generics* (generic method printing array).
- Design an **Enterprise Configuration Management System** applying these concepts.

Detailed Agenda (Time Slots & Activities)

Time Slot	Activity	Duration
9:00 – 9:15	Recap of Day 10 & Day 11 Overview – Advanced Concepts	15 min
9:15 – 9:40	Generics – Introduction, Generic Classes, Methods, Type Parameters	25 min
9:40 – 10:00	Bounded Type Parameters, Wildcards (<code>?</code> , <code>? extends</code> , <code>? super</code>)	20 min
10:00 – 10:15	Wrapper Classes – Conversion, Autoboxing/Unboxing	15 min
10:15 – 10:30	Immutable Class – Design Principles, Benefits	15 min
10:30 – 10:45	Tea Break	15 min

10:45 – 11:05	Singleton Design Pattern – Eager, Lazy, Thread-safe implementations	20 min
11:05 – 11:25	SOLID Principles – Overview of all 5 principles with examples	20 min
11:25 – 11:40	HackerRank: Java Generics Walkthrough	15 min
11:40 – 12:00	LeetCode: LRU Cache & Valid Sudoku (Approach Discussion)	20 min
12:00 – 12:30	Corporate Use Case: Enterprise Configuration Management System	30 min
12:30 – 12:50	Hands-on: Design a Singleton + Immutable Class	20 min
12:50 – 1:00	Homework & Closing	10 min

1. Generics

1.1 What are Generics?

- Generics enable **types (classes and interfaces)** to be parameters when defining classes, interfaces, and methods.
- Provides **compile-time type safety** – catches type mismatches at compile time.
- Eliminates the need for explicit casting.
- Introduced in Java 5.

1.2 Why Generics?

```
// Without Generics (pre-Java 5)
List list = new ArrayList();
list.add("Hello");
String s = (String) list.get(0); // Explicit casting required
list.add(123); // No compile-time error, but ClassCastException at runtime

// With Generics
List<String> list = new ArrayList<>();
list.add("Hello");
String s = list.get(0); // No casting needed
// list.add(123); // Compile-time error
```

1.3 Generic Classes

1.3 Generic Classes

```
public class Box<T> {
    private T content;

    public void set(T content) {
        this.content = content;
    }

    public T get() {
        return content;
    }
}

// Usage
Box<String> stringBox = new Box<>();
stringBox.set("Hello");
String value = stringBox.get();
```

1.4 Generic Methods

```
public <T> void printArray(T[] array) {
    for (T element : array) {
        System.out.print(element + " ");
    }
}
// Type parameter <T> appears before return type
```

1.5 Bounded Type Parameters

- Restrict the types that can be used as type arguments.
- Syntax: `<T extends Number>` – T must be Number or its subclass.
- `extends` is used for both classes and interfaces (not `implements`).

```
public <T extends Number> double sum(T a, T b) {
    return a.doubleValue() + b.doubleValue();
}
```

1.6 Wildcards (?)

- **Unbounded Wildcard:** `List<?>` – unknown type (can hold any List).
- **Upper Bounded Wildcard:** `List<? extends Number>` – Number or its subclasses.
- **Lower Bounded Wildcard:** `List<? super Integer>` – Integer or its superclasses.

Wildcard	Description	Example
----------	-------------	---------

<code>?</code>	Any type	<code>void print(List<?> list)</code>
<code>? extends T</code>	T or subtype	<code>void process(List<? extends Number> list)</code>
<code>? super T</code>	T or supertype	<code>void add(List<? super Integer> list)</code>

PECS Rule: Producer Extends, Consumer Super.

- If you **read** from a structure → use `extends`.
- If you **write** to a structure → use `super`.

1.7 Type Erasure

- Generics are a compile-time feature.
- At runtime, type information is erased (`List<String>` becomes `List`).
- This enables backward compatibility with non-generic code (pre-Java 5).

2. Wrapper Classes

2.1 What are Wrapper Classes?

- Each primitive type has a corresponding **wrapper class** in `java.lang`.
- Allows primitives to be used where objects are required (collections, generics).

Primitive	Wrapper Class
<code>byte</code>	<code>Byte</code>
<code>short</code>	<code>Short</code>
<code>int</code>	<code>Integer</code>
<code>long</code>	<code>Long</code>
<code>float</code>	<code>Float</code>
<code>double</code>	<code>Double</code>
<code>char</code>	<code>Character</code>
<code>boolean</code>	<code>Boolean</code>

2.2 Autoboxing and Unboxing

- **Autoboxing:** Automatic conversion of primitive to wrapper object.

```
Integer num = 42; // Autoboxing (int → Integer)
```

- **Unboxing:** Automatic conversion of wrapper to primitive.

```
int value = num; // Unboxing (Integer → int)
```

- Introduced in Java 5.

2.3 Common Methods

Method	Description
<code>valueOf(primitive)</code>	Returns wrapper object (cached for some values).
<code>parseXxx(String)</code>	Parses string to primitive (e.g., <code>Integer.parseInt("123")</code>).
<code>XxxValue()</code>	Returns primitive value from wrapper.
<code>compareTo()</code>	Compares wrapper objects.
<code>equals()</code>	Compares values (not reference).

2.4 Caching

- `Integer` , `Byte` , `Short` , `Long` cache values between -128 and 127.
- `==` works for cached values but not for others; use `equals()` .

2.5 Performance Considerations

- Autoboxing/unboxing in loops can create many objects → use primitives where possible.
- `Integer.valueOf(100)` uses cache; `new Integer(100)` does not (deprecated).

3. Immutable Class

3.1 What is Immutability?

- An object whose state **cannot be changed** after creation.
- Examples: `String` , `Integer` , `LocalDate` .
- Benefits: Thread-safe, cacheable, easy to reason about.

3.2 How to Design an Immutable Class?

Rule	Description
1. Declare class <code>final</code>	Prevents subclassing (which could modify fields).

2. Declare all fields <code>final</code> and <code>private</code>	Ensures they cannot be changed after construction.
3. No setter methods	No methods that modify fields.
4. Initialize all fields via constructor	Set values once, never change.
5. Return copies of mutable fields	If fields are mutable objects (e.g., Date, List), return defensive copies.

3.3 Example (Conceptual)

```
public final class ImmutablePerson {
    private final String name;
    private final List<String> hobbies;

    public ImmutablePerson(String name, List<String> hobbies) {
        this.name = name;
        this.hobbies = new ArrayList<>(hobbies); // Defensive copy
    }

    public String getName() {
        return name;
    }

    public List<String> getHobbies() {
        return new ArrayList<>(hobbies); // Defensive copy
    }
}
```

3.4 Benefits of Immutability

- Thread-safe (no synchronization needed).
- Can be safely shared across multiple threads.
- Hashcode is stable (good for HashMap keys).
- Prevents unintended side effects.

4. Singleton Design Pattern

4.1 What is Singleton?

- A creational design pattern that ensures a class has **only one instance** and provides a global access point to it.
- Used for: configuration managers, logging, thread pools, database connection pools.

4.2 Eager Initialization

```
public class SingletonEager {
    private static final SingletonEager instance = new SingletonEager();

    private SingletonEager() { } // Private constructor

    public static SingletonEager getInstance() {
        return instance;
    }
}
```

- **Pros:** Thread-safe, simple.
- **Cons:** Instance created even if not used (wasteful if expensive).

4.3 Lazy Initialization (Not Thread-safe)

```
public class SingletonLazy {
    private static SingletonLazy instance;

    private SingletonLazy() { }

    public static SingletonLazy getInstance() {
        if (instance == null) {
            instance = new SingletonLazy();
        }
        return instance;
    }
}
```

- **Pros:** Created only when needed.
- **Cons:** Not thread-safe (two threads could create two instances).

4.4 Thread-safe Lazy Initialization (Synchronized)

```
public class SingletonThreadSafe {
    private static SingletonThreadSafe instance;

    private SingletonThreadSafe() { }

    public static synchronized SingletonThreadSafe getInstance() {
        if (instance == null) {
            instance = new SingletonThreadSafe();
        }
        return instance;
    }
}
```

```
}
```

- **Pros:** Thread-safe.
- **Cons:** Synchronization overhead on every call (performance hit).

4.5 Double-Checked Locking (Optimized)

```
public class SingletonDCL {
    private static volatile SingletonDCL instance;

    private SingletonDCL() { }

    public static SingletonDCL getInstance() {
        if (instance == null) {
            synchronized (SingletonDCL.class) {
                if (instance == null) {
                    instance = new SingletonDCL();
                }
            }
        }
        return instance;
    }
}
```

- `volatile` ensures visibility across threads.
- **Pros:** Thread-safe, lazy, optimized (synchronization only first time).

4.6 Bill Pugh Singleton (Holder Pattern – Recommended)

```
public class SingletonBillPugh {
    private SingletonBillPugh() { }

    private static class Holder {
        private static final SingletonBillPugh INSTANCE = new SingletonBillPugh();
    }

    public static SingletonBillPugh getInstance() {
        return Holder.INSTANCE;
    }
}
```

- **Pros:** Thread-safe, lazy, no synchronization overhead.
- JVM ensures `Holder` class is loaded only when `getInstance()` is called.

4.7 Enum Singleton (Most Robust)

```
public enum SingletonEnum {
```

```
public enum SingletonEnum {  
    INSTANCE;  
    // Add methods here  
}
```

- **Pros:** Thread-safe, serialization-safe, reflection-proof.
- **Cons:** Less flexible (cannot extend another class).

5. SOLID Principles

5.1 What are SOLID Principles?

- Five design principles for writing maintainable and scalable object-oriented code.
- Introduced by Robert C. Martin (Uncle Bob).

5.2 S – Single Responsibility Principle (SRP)

- A class should have **one and only one reason to change**.
- **Example:** A `Employee` class should not handle both payroll and database operations.
- **Benefit:** Easier to maintain, test, and understand.

5.3 O – Open/Closed Principle (OCP)

- Software entities (classes, modules) should be **open for extension but closed for modification**.
- **Example:** Use inheritance or interfaces to add new behavior without changing existing code.
- **Benefit:** Reduces risk of breaking existing functionality.

5.4 L – Liskov Substitution Principle (LSP)

- Subtypes must be substitutable for their base types.
- If `S` is a subtype of `T`, objects of type `T` can be replaced with objects of type `S` without altering the program's correctness.
- **Example:** `Square` should not extend `Rectangle` if it changes behavior (area calculation).
- **Benefit:** Ensures correct inheritance hierarchies.

5.5 I – Interface Segregation Principle (ISP)

- Clients should not be forced to depend on interfaces they do not use.
- **Example:** Instead of a large `Worker` interface with `work()`, `eat()`, `sleep()`, split into `Workable`, `Eatable`, `Sleepable`.
- **Benefit:** Prevents unnecessary dependencies.

5.6 D – Dependency Inversion Principle (DIP)

- High-level modules should not depend on low-level modules; both should depend on abstractions.
- Abstractions should not depend on details; details should depend on abstractions.
- **Example:** `PaymentService` depends on `PaymentProcessor` interface, not on `CreditCardProcessor` or `PayPalProcessor`.
- **Benefit:** Loose coupling, easier to swap implementations.

5.7 SOLID in a Nutshell

Principle	Concept	Example
SRP	One reason to change	Separate data access from business logic
OCP	Extend without modifying	Use interfaces, strategy pattern
LSP	Substitutability	Square extends Shape, not Rectangle
ISP	Fat interface split	Separate <code>Printer</code> and <code>Scanner</code> interfaces
DIP	Depend on abstractions	<code>Service</code> depends on <code>Repository</code> interface

6. LeetCode Problems – Approach Discussion

6.1 LRU Cache (Least Recently Used Cache)

- **Problem:** Design a data structure that supports `get(key)` and `put(key, value)` in $O(1)$ average time. When cache reaches capacity, evict the least recently used item.
- **Approach:**
 - Use `LinkedHashMap` (which maintains insertion/access order).
 - Override `removeEldestEntry()` to enforce capacity.
 - Or use `HashMap` + custom `DoublyLinkedList` for full control.
- **Key concept:** `LinkedHashMap` with access order (`true`) naturally tracks LRU.
- **Relevance to Day 11:** Generics used in `HashMap<Integer, Integer>`, design pattern (cache eviction policy).

6.2 Valid Sudoku

- **Problem:** Determine if a 9×9 Sudoku board is valid (no duplicates in rows, columns, and 3×3 boxes).
- **Approach:**
 - Use `HashSet<String>` to track seen entries

Use `HashSet<String>` to track seen entries.

- For each cell, create row key: `row + " " + value`, column key: `col + " " + value`, box key: `box + " " + value`.
- If key already exists → invalid.

- **Relevance to Day 11:** HashSet uses hashing; generics with `Set<String>`.

7. HackerRank: Java Generics

- **Task:** Write a generic method `printArray(T[] array)` that prints all elements of an array of any type.
 - **Key learning:** Generic method syntax, type parameter `<T>`.
 - **Output:** Each element on a new line (`toString()` of each element).
-

8. Corporate Use Case: Enterprise Configuration Management System

8.1 Problem Context

A large enterprise needs a centralized configuration management system that:

- Loads configuration from multiple sources (properties files, environment variables, database).
- Provides a **single point of access** for all configuration (Singleton).
- Configuration values are **immutable** once loaded (cannot be changed at runtime).
- Supports **type-safe** retrieval of configuration values (Generics + Wrappers).
- Allows different configuration sources to be swapped easily (SOLID: DIP, OCP).
- Ensures thread-safe access from multiple application modules.

8.2 Application of Day 11 Topics

Requirement	Java Feature Used
Single configuration instance	Singleton (Bill Pugh Holder pattern)
Configuration values cannot change	Immutable <code>ConfigValue</code> class (final fields, no setters)
Type-safe retrieval	Generics – <code>getValue(String key, Class<T> type)</code>
Convert string to int/double/boolean	Wrapper Classes – <code>Integer.parseInt()</code> , <code>Double.parseDouble()</code>

Support multiple configuration sources	DIP – depend on <code>ConfigSource</code> interface; OCP – add new sources without changing core
Each source has single responsibility	SRP – separate classes for <code>FileSource</code> , <code>EnvSource</code> , <code>DbSource</code>
Interface segregation for sources	ISP – <code>ConfigSource</code> interface with <code>load()</code> method; no unnecessary methods
Liskov Substitution – all sources substitutable	LSP – any <code>ConfigSource</code> can be used; behavior consistent
Cache configuration values (optional)	Generics with <code>Map<String, ConfigValue<T>></code>
Thread-safe lazy loading	Singleton with thread-safe initialization

8.3 Example Architecture (Conceptual)

```

ConfigManager (Singleton)
├── ConfigSource (interface)
│   ├── FileConfigSource
│   ├── EnvConfigSource
│   └── DbConfigSource
├── ConfigValue<T> (immutable)
│   └── getValue(): T
└── getValue(key, Class<T>): T

```

8.4 Singleton Configuration Manager (Conceptual)

```

public final class ConfigManager {
    private static final class Holder {
        private static final ConfigManager INSTANCE = new ConfigManager();
    }

    private final Map<String, ConfigValue<?>> configMap = new HashMap<>();
    private final ConfigSource source;

    private ConfigManager() {
        this.source = new FileConfigSource(); // or from environment
        loadConfig();
    }

    public static ConfigManager getInstance() {
        return Holder.INSTANCE;
    }
}

```

```

private void loadConfig() {
    Map<String, String> raw = source.load();
    for (Map.Entry<String, String> entry : raw.entrySet()) {
        configMap.put(entry.getKey(), new ConfigValue<>(entry.getValue()));
    }
}

public <T> T getValue(String key, Class<T> type) {
    ConfigValue<> value = configMap.get(key);
    if (value == null) return null;
    return type.cast(value.getValue());
}
}

```

8.5 Immutable ConfigValue (Conceptual)

```

public final class ConfigValue<T> {
    private final T value;

    public ConfigValue(String raw, Class<T> type) {
        this.value = convert(raw, type);
    }

    public T getValue() {
        return value;
    }

    private T convert(String raw, Class<T> type) {
        if (type == String.class) return type.cast(raw);
        if (type == Integer.class) return type.cast(Integer.parseInt(raw));
        if (type == Double.class) return type.cast(Double.parseDouble(raw));
        if (type == Boolean.class) return type.cast(Boolean.parseBoolean(raw));
        throw new IllegalArgumentException("Unsupported type: " + type);
    }
}

```

8.6 SOLID in the Configuration System

SOLID Principle	How it's applied
SRP	Each <code>ConfigSource</code> handles one source (file, env, DB).
OCP	New sources (e.g., AWS Parameter Store) can be added without changing <code>ConfigManager</code> .
LSP	Any <code>ConfigSource</code> implementation can be used interchangeably.
ISP	<code>ConfigSource</code> interface has only <code>load()</code> – no extra methods.

DIP	<code>ConfigManager</code> depends on <code>ConfigSource</code> interface, not concrete classes.
-----	--

8.7 Discussion Questions

- Why make `ConfigManager` a Singleton? (Global access, single source of truth, avoids duplication.)
- Why make `ConfigValue` immutable? (Thread-safe, predictable behavior, prevents accidental modification.)
- How does Generics help here? (Type-safe retrieval – no casting required.)
- How would you add encryption/decryption without violating OCP? (Add a `DecoratorConfigSource` that wraps another source and decrypts.)

9. Hands-on Exercises (Without Code Lines)

9.1 Generic Class Design

Design a `Pair<K, V>` generic class with:

- Fields: `key` (K) and `value` (V).
- Constructor.
- Getters.
- `toString()` method.

9.2 Immutable Class Exercise

Identify what makes this class mutable and fix it:

- Class with `Date` field and getter returning the `Date` object directly.
- Class with `List` field and setter that adds elements.

9.3 Singleton Pattern – Which to Use?

When would you choose eager vs lazy vs Bill Pugh vs enum?

9.4 SOLID Violations

Identify which SOLID principle is violated:

- A class `ReportGenerator` handles PDF generation, HTML generation, and database fetching. (SRP)
- A `SmartDevice` interface has `print()`, `scan()`, `fax()` – but some devices can't fax. (ISP)
- High-level `OrderProcessor` directly depends on `MySQLDatabase`. (DIP)

9.5 Wrapper Class Practice

Given `String s = "123"`, how would you:

- Convert to `int` using wrapper class.
 - Convert to `Integer` object.
 - Check if the string is a valid integer without exception?
-

10. Quiz & Quick Check (Conceptual)

1. Which of the following uses Generics?

- (a) `ArrayList`
- (b) `ArrayList<String>`
- (c) `List list = new ArrayList()`
- (d) Both a and b

2. What is autoboxing?

- (a) Automatic conversion of primitive to wrapper
- (b) Automatic conversion of wrapper to primitive
- (c) Automatic conversion of one wrapper to another
- (d) Manual boxing using `new Integer(5)`

3. Which is NOT a requirement for an immutable class?

- (a) All fields are `final` and `private`
- (b) Class is `final`
- (c) All fields have setters
- (d) Defensive copies for mutable fields

4. Which Singleton implementation is thread-safe and lazy without synchronization overhead?

- (a) Eager initialization
- (b) Lazy initialization
- (c) Bill Pugh Holder pattern
- (d) Double-checked locking

5. Which SOLID principle says "depend on abstractions, not concrete implementations"?

- (a) SRP

- (b) OCP
- (c) LSP
- (d) DIP

Answers: 1-b, 2-a, 3-c, 4-c, 5-d

11. Additional Resources & Cheat Sheets

- **Generics Cheat Sheet:** `T` for type, `?` for unknown, `extends` for bounds.
- **Wrapper Class Methods:** `valueOf()`, `parseXxx()`, `xxxValue()`.
- **Immutable Class Checklist:** 5 rules to follow.
- **Singleton Patterns Comparison Table:** Eager, Lazy, Synchronized, DCL, Bill Pugh, Enum.
- **SOLID Principles Memory Aid:** "A class should have one reason to change (SRP), be open for extension (OCP), substitutable (LSP), segregated (ISP), and depend on abstractions (DIP)."

12. Closing Summary

Day 11 covered advanced Java concepts essential for building robust, maintainable enterprise applications. **Generics** provided type safety and eliminated casting. **Wrapper Classes** enabled primitives to be used in collections and generic code. **Immutable Classes** were designed to be thread-safe and predictable. The **Singleton Design Pattern** ensured a single instance of configuration managers and loggers. Finally, the **SOLID Principles** provided a foundation for writing clean, extensible, and maintainable code. The Enterprise Configuration Management System case study integrated all these concepts: Singleton for global access, Generics for type-safe retrieval, Immutability for thread-safe configuration values, and SOLID for flexible source management. LeetCode problems (LRU Cache, Valid Sudoku) and HackerRank (Java Generics) reinforced practical application.

13. Key Takeaways:

- Generics provide compile-time type safety and reduce casting.
- Wrapper classes enable primitives to be used in object contexts.
- Immutable classes are thread-safe and easier to reason about.
- Singleton ensures a single instance of a class.
- SOLID principles guide maintainable software design.

End of Day 11